DevOps Explanation Request

Absolutely, vicky 😈
Let's go **slow, surgical, and hands-on** — zero magic, zero assumptions.

I'll explain **EVERY command, WHAT it does, WHAT files it creates, WHERE they are created**, and **HOW to identify private keys vs certificates**.

🧠 Golden Rule (lock this first)

👉 **If a file name or extension contains `key`**
➡️ **It is a PRIVATE KEY** 🔐

👉 **If a file name or extension contains `crt`, `cert`, or `.pem` (cert part)**
➡️ **It is a CERTIFICATE** 📜

👉 **CSR (`.csr`) is NOT a certificate**
➡️ It is a **request** to get a certificate

☠️ Way (Manual Identity Without TLS)

- Anyone can pretend to be anyone
- No encryption
- Kubernetes refuses to trust you

So Kubernetes says:

> "Bring a certificate signed by my CA, or don't talk to me."

🛠️ How Kubernetes Solves This

Using **TLS Certificates** created via **CSR (Certificate Signing Request)**.

Now let's DO IT.

Step-by-Step Hands-on Example (ZERO CONFUSION)

We will create a **client certificate for a user named** `adam`

📍 **All files are generated in the CURRENT directory**

🟢 STEP 1: Generate a PRIVATE KEY

Command

```bash
openssl genrsa -out adam.key 2048
```

What this command does

- Generates a **2048-bit RSA private key**
- This key is **adam's secret identity**
- Used to sign requests and prove identity

Files created

File Name	Type	Meaning
`adam.key`	🔐 PRIVATE KEY	Adam's private key

✅ Contains `key` → PRIVATE KEY
❌ NOT shared with anyone

❌ NOT uploaded to Kubernetes

🧠 Important

Private key is created **FIRST**

Everything else depends on it.

🟢 STEP 2: Generate CSR (Certificate Signing Request)

Command

```bash
openssl req -new -key adam.key -out adam.csr -subj "/CN=adam"
```

What this command does

- Uses **adam.key** to create a **CSR**
- CSR contains:
  - Adam's public key
  - Adam's identity (`CN=adam`)
- This is a **request**, not a certificate

Files created

File Name	Type	Meaning
`adam.csr`	📩 CSR	Request to CA for certificate

⚠️ **CSR is NOT a certificate**

⚠️ Kubernetes does NOT trust CSR directly

Why `/CN=adam` matters

- `CN` = **Common Name**
- Kubernetes treats this as **username**
- RBAC permissions map to this name

🟢 STEP 3: Encode CSR (Required by Kubernetes)

Kubernetes expects CSR in **base64**

Command

```bash
cat adam.csr | base64 | tr -d "\n"
```

What this command does

- Reads `adam.csr`
- Converts it to base64
- Removes newlines (K8s requirement)

📌 Output will be pasted into YAML

❌ No new file created
✔️ Just terminal output

🟢 STEP 4: Create Kubernetes CSR Object

File: `adam-csr.yaml`

```yaml
```

```yaml
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: adam-csr
spec:
  request: <BASE64_CSR_HERE>
  signerName: kubernetes.io/kube-apiserver-client
  usages:
  - client auth
```

What this does

- Submits Adam's CSR to Kubernetes CA
- Requests a **client certificate**
- Kubernetes stores it but does NOT approve it yet

Apply it

```bash
kubectl apply -f adam-csr.yaml
```

Result

- CSR object created in cluster
- Status = `Pending`

Check:

```bash
kubectl get csr
```

🟢 STEP 5: Approve the CSR

Command

```bash
kubectl certificate approve adam-csr
```

What this does

- Kubernetes CA signs the CSR
- Generates a **certificate**
- Attaches cert to CSR object

OR ❌ Deny it:

```bash
kubectl certificate deny adam-csr
```

🟢 STEP 6: Extract the SIGNED CERTIFICATE

After approval, Kubernetes stores the cert inside CSR object.

Command

```bash
kubectl get csr adam-csr -o jsonpath='{.status.certificate}' \
| base64 --decode > adam.crt
```

What this command does

- Reads signed certificate from Kubernetes
- Decodes it

- Saves it to a file

Files created

File Name	Type	Meaning
`adam.crt`	📜 CERTIFICATE	Signed client certificate

✅ Contains `crt` → CERTIFICATE
✅ Trusted by Kubernetes CA

📂 Final Files in Your Directory

```text
adam.key  → PRIVATE KEY 🔐
adam.csr  → CSR (request) 📩
adam.crt  → CERTIFICATE 📜
```

Classification (VERY IMPORTANT)

File	What it is
`*.key`	PRIVATE KEY
`*.csr`	Certificate request
`*.crt / *.cert`	CERTIFICATE

🧩 How Kubernetes Uses These Files

- `adam.key` → proves identity

- `adam.crt` → trusted by Kubernetes
- Both are referenced in **kubeconfig**

Without **both**, authentication FAILS.

Step 7 – Common Mistakes (Seen in real clusters)

❌ Uploading `.key` to GitHub
❌ Thinking CSR = certificate
❌ Missing `CN` → user becomes `unknown`
❌ Forgetting to approve CSR
❌ Losing private key → cert useless

🧠 Final Mental Model (Tattoo this)

```pgsql
.key  → Who I am (SECRET)
.csr  → Please trust me
.crt  → You are trusted
```

If you want next:

- kubeconfig file mapping (key + cert)
- TLS flow between API server & kubelet
- Cert expiry & rotation
- Debugging x509 errors (real logs)

Say the word 😏🔥